

LLM-Powered SQL Query Agent (FastAPI + Groq + SQLite)

Introduction

The LLM-Powered SQL Query Agent is an AI-driven system that allows users to query structured databases using natural language. Instead of writing SQL queries manually, users can ask questions in plain English, and the system automatically converts them into optimized SQL statements using a Large Language Model (LLaMA 3 via Groq API).

This project demonstrates the integration of:

- Generative AI
- Backend API development
- Secure SQL validation
- Database management

The system ensures that only safe and read-only queries are executed, preventing data corruption and SQL injection attacks.

Problem Statement

Traditional database querying requires:

- Knowledge of SQL syntax
- Understanding of table relationships
- Ability to write joins and aggregations

Non-technical users struggle to extract insights from databases.

There is a need for an intelligent system that:

- Understands natural language
- Converts it into SQL queries
- Ensures security
- Returns structured results

Objectives

The primary objectives of this project are:

- Convert natural language questions into SQL queries
- Automatically extract database schema
- Validate SQL queries for security

- Execute safe queries on SQLite database
- Return formatted and structured responses
- Prevent SQL injection and destructive operation

System Architecture

Architecture Flow

User



FastAPI (API Layer)



Agent Pipeline



Groq LLM (LLaMA 3.3-70B)



SQL Validator



SQLite Database



Formatted JSON Response

Technology Stack

Component	Technology Used
Backend API	FastAPI
LLM Model	LLaMA 3.3-70B (Groq API)
Database	SQLite (Chinook Dataset)
Programming Language	Python 3.10+
Environment Management	python-dotenv
Web Server	Uvicorn

Working Methodology

Step 1 – Schema Extraction

Function:

`get_schema()`

This function:

- Retrieves table names from SQLite
- Extracts column names
- Formats schema into readable text
- Sends schema context to LLM

Purpose:

Ensures the LLM understands table relationships before generating SQL.

Step 2 – SQL Generation Using LLM

Function:

`generate_sql(question, schema)`

The system sends:

- User question
- Database schema
- Instruction to generate only SELECT queries

Model used:

llama-3.3-70b-versatile

The LLM generates optimized SQL based on schema context.

Step 3 – SQL Validation

Function:

`validate_sql(query)`

Security checks include:

✓ Only SELECT queries allowed

✓ Blocks:

- DROP

- DELETE
- INSERT
- UPDATE
- ALTER
- TRUNCATE

Purpose:

Prevent destructive operations and SQL injection.

Step 4 – SQL Execution

Function:

`execute_sql(query)`

- Connects to SQLite database
- Executes validated SQL
- Fetches results
- Converts output to JSON format

API Documentation

Endpoint

GET /ask

Example Request

`http://127.0.0.1:8000/ask?question=How many tracks are in the Rock genre?`

Example Response

```
{
  "response": {
    "question": "How many tracks are in the Rock genre?",
    "sql_query": "SELECT COUNT(*) FROM Track t JOIN Genre g ON t.GenreId = g.GenreId\nWHERE g.Name = 'Rock';",
    "answer": "COUNT(*): 1297"
  }
}
```

Output screenshots

Ask Question

How many tracks are in the Rock genre?

Submit

Question: How many tracks are in the Rock genre?

SQL Query: `SELECT COUNT(*) FROM Track t JOIN Genre g ON t.GenreId = g.GenreId WHERE g.Name = 'Rock';`

Answer: 1297

Ask Question

What are the top 3 genres by number of tracks?

Submit

Question: What are the top 3 genres by number of tracks?

SQL Generated:
`SELECT g.Name as Genre, COUNT(t.TrackId) as TrackCount
FROM genres g
JOIN tracks t ON g.GenreId = t.GenreId
GROUP BY g.GenreId
ORDER BY TrackCount DESC
LIMIT 3;`

Final Answer:
The top 3 genres are Rock (1,297), Latin (579), Metal (374).